

Plan de trabajo

Integración de estructuras de datos en Control de Gastos

Documento operativo para que la aplicación cumpla los seis requisitos obligatorios: lista enlazada, pila, cola FIFO, árbol binario, tabla hash y grafo. Cada estructura debe tener implementación propia y un propósito real en el producto.

Estado de esta entrega

Fase 1 completada: la lista enlazada es la colección principal de movimientos, con insertar, eliminar, buscar y recorrer conectados a la app y a Supabase.

Aplicación: Control de Gastos · Expo + React Native + TypeScript

Repositorio: App-ControlGastos

1. Objetivo del plan

Definir el orden de trabajo, las responsabilidades técnicas y los criterios de aceptación para incorporar las seis estructuras de datos exigidas, sin romper el flujo actual de autenticación, navegación, Redux y sincronización con Supabase.

El resultado esperado no es un conjunto de clases aisladas. Cada estructura debe intervenir en un flujo que el usuario (o el docente en la demo) pueda observar: crear, buscar, deshacer, procesar en orden, clasificar, localizar por clave o recorrer relaciones.

2. Diagnóstico inicial

Antes de este plan, el dominio financiero se modelaba así:

- **Colecciones**

Cuentas, movimientos y metas vivían en arrays nativos (`Account[]`, `MovementItem[]`, `SavingsMeta[]`).

- **Cachés**

Redux usaba `Record<string, MovementItem[]>` para agrupar por mes y por cuenta. Eso no demuestra tabla hash propia ni manejo de colisiones.

- **Navegación**

React Navigation Stack organiza pantallas. No es una pila LIFO de dominio (deshacer, historial de acciones o estados).

- **Vacío total**

No existían cola FIFO, árbol binario ni grafo. Tampoco había eliminar movimiento, operación indispensable para la lista enlazada.

Regla de oro: un Array, un objeto de JavaScript o el Stack de React Navigation no sustituyen la implementación académica exigida.

3. Principios de implementación

01 Implementación propia

Clases TypeScript en `src/structures/`. Nodos, punteros o cubetas deben ser visibles en el código.

02 Propósito real

La estructura opera un flujo del producto. No basta con crearla y no usarla.

03 Operaciones completas

Cada requisito lista operaciones mínimas. Si pide insertar, eliminar, buscar y recorrer, las cuatro deben ejecutarse desde la app.

04 Conviven con Redux y Supabase

La estructura es la fuente de verdad en memoria. Redux guarda un snapshot para pintar la UI. Supabase persiste en la nube.

05 Demo defendible

Al final de cada fase debe existir un recorrido oral de 60–90 segundos que muestre para qué sirve la estructura.

4. Flujo general de actividades

El trabajo se ejecuta en serie. Cada fase cierra con código integrado, prueba manual y evidencia para la presentación.

Fase	Estructura	Resultado visible	Estado
0	Preparación	Carpeta src/structures y reglas de integración	Hecha
1	Lista enlazada	CRUD de movimientos sobre nodos	Hecha
2	Pila LIFO	Deshacer crear, editar o borrar	Pendiente
3	Cola FIFO	Pagos o sincronización en orden de llegada	Pendiente
4	Árbol binario	Ranking o catálogo con recorridos	Pendiente
5	Tabla hash	Búsqueda por clave con colisiones	Pendiente
6	Grafo	Relaciones entre cuentas o metas + BFS/DFS	Pendiente
7	Cierre	Demo, FAQ y criterios de aceptación	Pendiente

Secuencia recomendada de un ciclo de fase:

- **Diseñar el uso**
Elegir el flujo de negocio que justifica la estructura dentro de Control de Gastos.
- **Implementar la clase**
Operaciones, tipos genéricos y casos borde (vacía, un elemento, no encontrado).
- **Conectar el dominio**
El servicio o slice llama a la estructura; la UI no opera arrays “en paralelo”.
- **Exponer en pantalla**
El usuario dispara insertar, eliminar, buscar, deshacer, encolar o recorrer.
- **Verificar**
Probar el flujo feliz, un error y el cierre de sesión. Confirmar que no quedan datos de otro usuario.
- **Documentar para defensa**
Una frase de propósito + dónde está el código + qué se ve en la demo.

5. Fase 0 — Preparación

Crear src/structures/ como único lugar de las estructuras. Redux no debe guardar nodos con punteros: Immer y la serialización no son adecuadas para eso. El patrón acordado es: la clase vive en un repositorio en memoria; tras cada mutación se recorre la estructura y se actualiza el snapshot de Redux para la UI.

Archivos base: ListaEnlazada.ts, movimientosLista.ts e index.ts. Las fases siguientes añaden Pila.ts, Cola.ts, ArbolBinario.ts, TablaHash.ts y Grafo.ts.

6. Fase 1 — Lista enlazada (completada)

Propósito obligatorio: almacenar la colección principal de objetos, con insertar, eliminar, buscar y recorrer.

Decisión: los movimientos son la colección principal de la app (Inicio, detalle de cuenta, estadísticas y formulario). Por eso la lista enlazada es la fuente de verdad en memoria de `MovementItem`.

- **Clase**

`ListaEnlazada<T>` con `Nodo` (valor + siguiente). Métodos: `insertar`, `insertarAlInicio`, `insertarAlFinal`, `eliminar`, `eliminarTodos`, `buscar`, `recorrer` y `vaciar`.

- **Repositorio**

`movimientosLista` envuelve la lista: `insertar` un movimiento, eliminarlo por id, buscarlo por id, recorrer todos, reemplazar un mes y fusionar los de una cuenta.

- **Insertar**

Al crear un movimiento (`addMovimientoThunk`) o al actualizarlo. También al cargar el mes o la cuenta desde Supabase.

- **Eliminar**

Nuevo `deleteMovimientoThunk` + botón Eliminar en Editar registro. Quita el nodo y borra la fila en Supabase.

- **Buscar**

`selectMovimientoById` usa `movimientosLista.buscar(id)` para abrir el formulario de edición.

- **Recorrer**

`construirSnapshotMovimientos` recorre la lista y arma las vistas por mes y por cuenta que pinta la UI.

Cierre de sesión: `resetFinanceState` vacía la lista para que el siguiente usuario no herede nodos ajenos. Esto es parte del propósito real, no un detalle opcional.

Actividades de cierre de esta fase:

- [] Crear un gasto y verlo en Inicio (insertar + recorrer).
- [] Abrir ese gasto (buscar por id) y editarlo.
- [] Eliminarlo y confirmar que desaparece de Inicio y de la cuenta.
- [] Cerrar sesión y entrar con otra cuenta: la lista debe nacer vacía.

7. Fase 2 — Pila (LIFO)

Propósito: historial deshacer/rehacer, registro de acciones o control de estados. El Stack de navegación no cuenta.

Actividades:

1 Implementar `Pila<T>`

`push`, `pop`, `peek` e `isEmpty`. Opcional: pila de rehacer.

2 Modelar la acción

Tipo `HistorialAccion` con crear, editar y eliminar movimiento, más el snapshot necesario para revertir.

3 Empujar al guardar

Tras insertar, actualizar o eliminar con éxito, hacer `push` de la acción inversa.

4 Botón Deshacer

En Inicio o en el formulario. `pop` + aplicar la acción inversa sobre la lista enlazada y Supabase.

5 Límite y limpieza

Tope de N acciones y vaciar la pila al cerrar sesión.

Criterio de hecho: el usuario crea un gasto, pulsa Deshacer y el gasto desaparece; si había edición, vuelve el valor anterior.

8. Fase 3 — Cola FIFO

Propósito: turnos, procesamiento de tareas o solicitudes pendientes. First in, first out.

Uso recomendado en este producto (elegir uno y llevarlo hasta la UI):

- **Opción A — Pagos programados**

Los vencimientos se encolan por fecha. El frente de la cola es el siguiente pago a atender.

- **Opción B — Sincronización**

Si una operación falla por red, se encola y se procesa en orden al reconectar.

Actividades:

- 1 **Implementar Cola<T>**

enqueue, dequeue, front e isEmpty.

- 2 **Alimentar la cola**

Desde movimientos con dueDate o desde fallos de thunks.

- 3 **Procesar el frente**

Botón o efecto que atiende el primer elemento y lo saca de la cola.

- 4 **Mostrar pendientes**

Lista “en cola” para que el FIFO se vea, no solo se infiera.

Criterio de hecho: el orden de salida coincide con el de llegada; no se atiende un elemento posterior antes que el frente.

9. Fase 4 — Árbol binario

Propósito: organización jerárquica o búsqueda eficiente. Debe insertar, buscar e inorden / preorden / postorden.

Uso recomendado: árbol de búsqueda por monto (movimientos) o por montoObjetivo / prioridad (metas). El inorden entrega el ranking ordenado.

Actividades:

- 1 **Implementar BST**

Nodo con izquierdo y derecho. insertar, buscar, inorden, preorden y postorden.

- 2 **Reconstruir al recorrer la lista**

Tras sincronizar movimientos, armar el árbol desde la lista enlazada. Así lista y árbol colaboran.

- 3 **Pantalla o sección Ranking**

Mostrar al menos un recorrido (inorden = menor a mayor gasto) y poder buscar un monto.

- 4 **Defensa oral**

Explicar por qué inorden ordena, y qué muestran preorden y postorden.

Criterio de hecho: insertar un movimiento cambia el ranking; buscar por clave del árbol encuentra el nodo; los tres recorridos están implementados y uno se ve en UI.

10. Fase 5 — Tabla hash

Propósito: búsqueda rápida por clave, con manejo explícito de colisiones. Record o Map nativos no bastan.

Uso recomendado: diccionario id → movimiento (y, si aplica, email → sesión local). selectMovimientoById puede pasar de la lista lineal a la tabla cuando esta fase cierre, o convivir: la lista es la colección, la hash es el índice.

Actividades:

1 Implementar TablaHash<K, V>

Función hash propia, arreglo de cubetas y colisiones por encadenamiento (lista por bucket) o sondeo.

2 Operaciones

set, get y remove. Demostrar dos claves distintas que caigan en el mismo bucket.

3 Indexar al insertar/eliminar

Cada mutación de la lista enlazada actualiza la tabla.

4 Usar get en búsqueda

Editar movimiento y cualquier lookup por id deben pasar por la tabla.

Criterio de hecho: hay código de colisiones; una búsqueda por id no recorre toda la colección; se puede explicar un ejemplo de colisión.

11. Fase 6 — Grafo

Propósito: relaciones o conexiones. Lista o matriz de adyacencia, más BFS o DFS.

Uso recomendado (elegir uno):

- **Cuentas**
Aristas = transferencias o “paga desde”. BFS encuentra una ruta entre dos cuentas.
- **Metas**
Aristas = dependencias (hay que completar A antes que B). DFS detecta el orden o un ciclo.
- **Categorías**
Red de categorías relacionadas para sugerir clasificación.

Actividades:

1 Implementar Grafo

Lista de adyacencia (preferible) o matriz. agregarVertice, agregarArista, vecinos.

2 BFS o DFS

Al menos un recorrido completo, devolviendo el orden visitado.

3 Cargar desde datos reales

Vértices = cuentas o metas del usuario, no un ejemplo hardcodeado ajeno a la app.

4 UI de relaciones

Pantalla o bloque que muestre conexiones y el resultado de BFS/DFS (“ruta sugerida”, “orden de metas”).

Criterio de hecho: el grafo se construye con datos del usuario; el recorrido se dispara desde la UI y el orden visitado es visible.

12. Fase 7 — Integración, demo y defensa

Cuando las seis estructuras estén vivas, unificar el relato de presentación:

- **Mapa mental**
Lista = colección de movimientos. Pila = deshacer. Cola = pendientes. Árbol = ranking. Hash = índice por id. Grafo = relaciones.
- **Guion de 8 minutos**
Login → crear gasto (lista) → buscar/editar → deshacer (pila) → ver cola → ranking (árbol) → lookup (hash) → recorrido (grafo).
- **FAQ**
Actualizar FAQ_PRESENTACION.md con una sección de estructuras: por qué no se usó Array, dónde está cada clase, cómo se demuestra el propósito.
- **Limpieza**
Vaciar todas las estructuras al signOut. No dejar nodos, pilas ni colas de un usuario en el siguiente.

13. Dependencias entre fases

La lista enlazada es la base: pila, cola, árbol y hash se alimentan de mutaciones sobre movimientos. Por eso el orden 1 → 2 → 3 → 4 → 5 es el de menor retrabajo. El grafo puede avanzar en paralelo a partir de cuentas y metas,

sin esperar el árbol, pero debe respetar el mismo patrón de repositorio en memoria + snapshot de UI.

Bloque	Se apoya en	No debe hacerse antes de
Pila de deshacer	Insertar / eliminar de la lista	Fase 1
Cola de pendientes	Movimientos con vencimiento o thunks	Fase 1
Árbol de ranking	Recorrer la lista para reconstruir	Fase 1
Tabla hash índice	Ids de la lista; set/remove al mutar	Fase 1
Grafo de cuentas	Cuentas (y opcionalmente movimientos)	Fase 0

14. Criterios de aceptación globales

- [] Existen seis clases propias, no wrappers de Array/Map.
- [] Cada clase se usa en un flujo real; ninguna queda solo exportada.
- [] Lista: insertar, eliminar, buscar y recorrer se ejecutan desde la app.
- [] Pila: un Deshacer restaura el estado anterior (LIFO).
- [] Cola: el primero en entrar es el primero en atenderse (FIFO).
- [] Árbol: insertar, buscar e inorden, preorden y postorden.
- [] Hash: función hash propia y estrategia de colisiones visible.
- [] Grafo: adyacencia + BFS o DFS sobre datos del usuario.
- [] Cerrar sesión limpia todas las estructuras en memoria.
- [] Supabase sigue siendo la persistencia; las estructuras no reemplazan la base de datos.
- [] La UI existente (Inicio, Cuentas, Metas, Login) no pierde comportamiento.

15. Entregables por fase

Fase	Código	Evidencia de demo
1	ListaEnlazada + movimientosLista + delete	Crear, abrir, borrar un movimiento
2	Pila + botón Deshacer	Crear y deshacer en un gesto
3	Cola + lista de pendientes	Dos elementos salen en orden
4	BST + ranking	Inorden visible; búsqueda en árbol
5	TablaHash + get por id	Explicar una colisión
6	Grafo + BFS/DFS	Ruta o dependencias en pantalla
7	FAQ y guion	Defensa continua de 8 minutos

16. Riesgos y cómo atajarlos

• Redux + punteros

No guardar nodos en el store. Snapshot vía `recorrer()` después de cada mutación.

• Estructura “de adorno”

Si la UI sigue usando un array paralelo que no pasa por la clase, el requisito no se cumple. Toda mutación entra por el repositorio.

• Datos de otro usuario

`vaciar()` en `signOut` para lista, pila, cola, árbol, hash y grafo.

• Confundir Stack de navegación con pila

Dejarlo explícito en la defensa: React Navigation no es el requisito 2.

• Hash nativo

`Record` y `Map` no demuestran colisiones. La clase debe tener cubetas y una estrategia visible.

• Grafo de juguete

No usar ciudades inventadas. Vértices = cuentas, metas o categorías del usuario.

17. Cómo se ve el flujo en la práctica

Usuario autenticado → Inicio pide el mes → Supabase devuelve filas → cada movimiento se inserta en la lista enlazada → recorrer() arma el snapshot de Redux → la pantalla pinta tarjetas. El usuario abre un movimiento → buscar(id) carga el formulario. Guarda cambios → se actualiza el nodo y Supabase. Elimina → se desconecta el nodo y se borra en la nube. Fases 2 a 6 se enganchan en esos mismos puntos: la pila observa la mutación, la cola observa vencimientos, el árbol y la hash se reconstruyen o actualizan al recorrer, el grafo se arma con cuentas y relaciones.

Siguiente paso inmediato

Verificar manualmente la Fase 1 (crear, buscar, eliminar, logout) y continuar con la Fase 2: pila LIFO de deshacer sobre las mismas mutaciones de movimientos. No iniciar el árbol ni el grafo hasta que Deshacer funcione; así se evita rehacer la integración dos veces.